

Seven AOP theorems, drawn

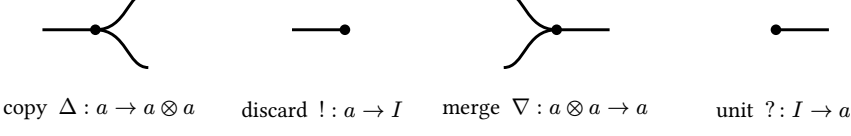
primitives in `diag/strdiag.typ`; the calculus is `functorialSemanticsForRelationalTheories.pdf §4`

A string diagram is an arrow of the monoidal category drawn as a circuit. Left-to-right juxtaposition is composition $\gg$ — the book’s diagram order, so $R \ S$ means first R then S — and vertical stacking is the monoidal product. A wire is an identity, a white box is a relation, and the black dots are one algebraic structure: the special Frobenius (co)monoid every object carries in a cartesian bicategory of relations. `diag/S2_124.typ` is the companion note that proves a theorem in this calculus rather than only displaying one.

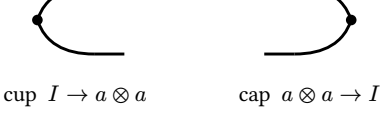
Each picture below is a Lean statement from this repo, drawn field for field. Converse is written R° , as the Lean does; `functorialSemanticsForRelationalTheories.pdf` writes the same operation $R\ddagger$.

0. The vocabulary

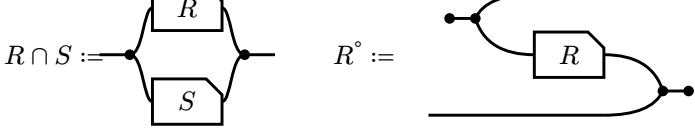
The four dots are Δ (copy), $!$ (discard), ∇ (merge), $?$ (unit) — Definition 4.1, clauses 1 and 2, carried by `diag/CB.lean` as the `CartBicat` fields `delta/bang/nabla/unitR`.



Bending a wire is done with a **cup** and a **cap** — $?$; Δ and ∇ ; $!$, whose dots the Frobenius equations collapse. Bending down and back up again straightens out, which is what lets a wire be turned around at all (p. 19).



Two derived pieces are used in every diagram that follows, so they come first. The **meet** $R \cap S$ is not a new generator: it is the convolution $\Delta \gg (R \otimes S) \gg \nabla$ (p. 22, `meet` in `diag/CB_Derived.lean`). Copy the input, run R and S on the two copies, then merge — and the merge forces the two results to coincide, which is exactly “both R and S ”. The **converse** R° is not a new generator either: bend both of R ’s wires around, and what was an input is read as an output (p. 19, `conv` in `diag/CB.lean`).



Everywhere below, R° is drawn as a plain box rather than as that bent picture — the bending is what R° is, not something to redraw at every occurrence.

What the pictures mean in Rel(Set)

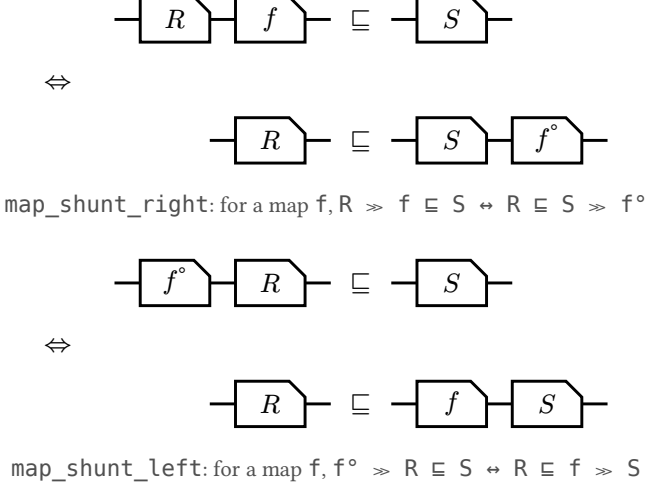
The captions below are allowed to name these shapes $\cap$ and $^\circ$ only because `diag/RelSetCB.lean` proves they **are** those operations in this repo’s own `Rel(Set)`, not merely something that behaves like them. That file gives the whole dictionary:

drawn	what it is in Rel(Set), and the theorem that says so
a copy dot Δ	<code>deltaRel</code> $\cap$ = <code>graph (fun x => (x, x))</code> , the diagonal — and <code>delta_eq</code>
a discard dot $!$	<code>bangRel</code> $\cap$ = <code>graph (fun _ => PUnit.unit)</code> , the map to a point — <code>bang_eq</code>
a merge dot ∇	<code>nabla_eq</code> : $\nabla = \Delta^\circ$; <code>deltaRel_recip_apply</code> says $(x, y) \nabla z \leftrightarrow x = z \wedge y = z$, which is why merging two strands demands that both agree
a unit dot $?$	<code>unitR_eq</code> : $? = !^\circ$; <code>bangRel_recip_apply</code> says it relates the point to everything
the meet combinator	<code>meet_eq_inter</code> : $\Delta \gg (R \otimes S) \gg \nabla = R \cap S$, the intersection of <code>Allegory RelSet</code>
a bent-wire converse	<code>conv_eq_recip</code> : <code>conv</code> $R = R^\circ$, the ordinary relational converse
$! \gg ?$	<code>top_apply</code> : $\top$ relates everything to everything

One thing every picture here silently does that its Lean statement does not: it **suppresses the coherence maps**. Stacking three wires and re-bracketing them is free on paper, but $(a \times b) \times c$ is not $a \times (b \times c)$ as a Lean type, so `diag/Monoidal.lean` carries a real associator arrow `tensAssoc`, spelled out in `delta_assoc` and in both halves of the Frobenius equation `frob_left/frob_right`. Wherever a diagram below stacks more than two strands — the modular law and the tabulation `meet` — read it as the picture of a statement that has associators written out.

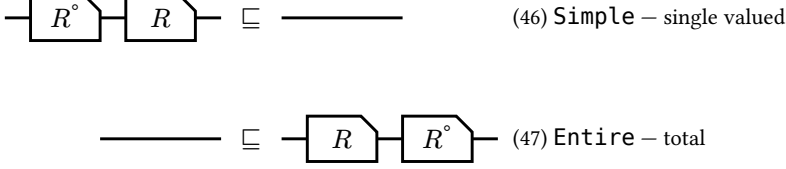
1. Map shunting — `map_shunt_right`, `map_shunt_left`

The workhorse of relational program calculation (AOP/A4_2.lean, B&dM 4.19 and 4.20): for a **map** f , composing with f on one side of a containment is the same as composing with f° on the other. Diagrammatically this is the unit/counit triangle of the adjunction $f \dashv f^\circ$ — Lemma 4.8 of `functorialSemanticsForRelationalTheories.pdf`, which says f is a map exactly when it has a right adjoint, and that adjoint is f° .

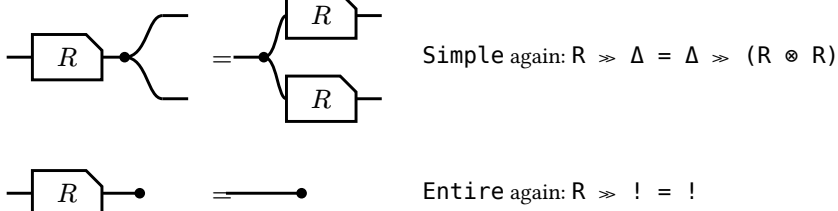


2. The map dictionary — Entire, Simple, Map

`Freyd/S2_10.lean §2.13` defines `Simple` $R := R^\circ \gg R \sqsubseteq 1$ and `Entire` $R := \text{dom } R = 1$, the latter equivalent to $1 \sqsubseteq R \gg R^\circ$; a `Map` is both. These are inequations (46) and (47) of Lemma 4.4, p. 20, where the paper calls them single-valued and total.

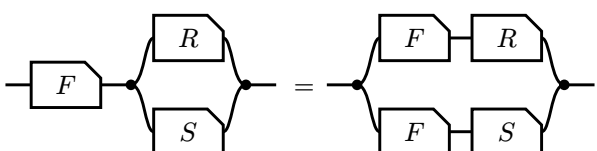


The same two conditions read off the Frobenius structure instead. Every arrow is only a **lax** comonoid homomorphism — that is eq. (3), p. 4, the `lax_Δ` and `lax_!` fields — and a map is precisely an arrow for which those two inequations are equalities.



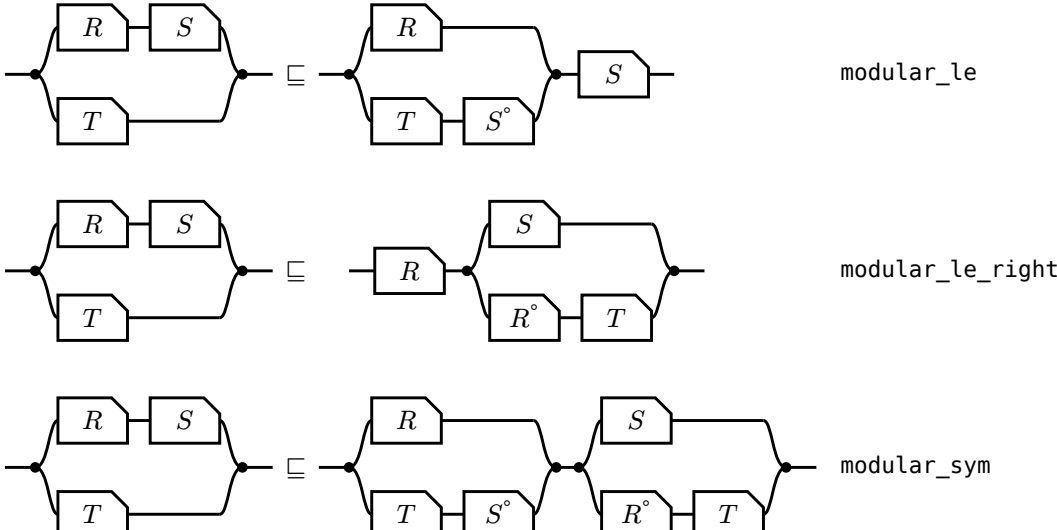
3. Simple relations distribute over the meet — `simple_dist_inter`

`Freyd/S2_10.lean §2.136`: if F is simple then $F \gg (R \cap S) = (F \gg R) \cap (F \gg S)$. The $\sqsubseteq$ direction of this is eq. (3) on p. 4 — it holds for every arrow, because F can always be copied early rather than late. Simplicity is exactly what tightens it to an equality, which is why this is the engine of §2.141 below.



4. The modular law — `modular_le`, `modular_le_right`, `modular_sym`

The one axiom of an allegory that is not a lattice law (`Freyd/S2_10.lean §2.11`, order form `modular_le`). Its two one-sided halves are `modular_le_right` (AOP/A4_1.lean), and B&dM 4.8’s symmetric form `modular_sym` is visibly the two halves applied at once.

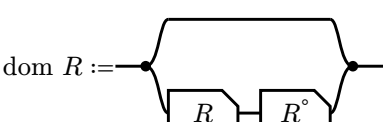


5. Domain — `dom`, `dom_UP`

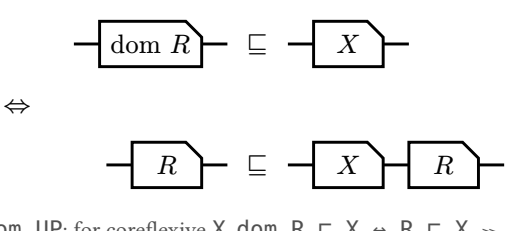
A relation $X : a \rightarrow a$ is **coreflexive** when $X \sqsubseteq 1$: a wire that filters and does nothing else.



`dom` R (`Freyd/S2_10.lean §2.122`) is the coreflexive $1 \cap R \gg R^\circ$ — go out along R and come back, and keep only the inputs for which that round trip returns. The identity strand of the meet is drawn as what it is, a bare wire.

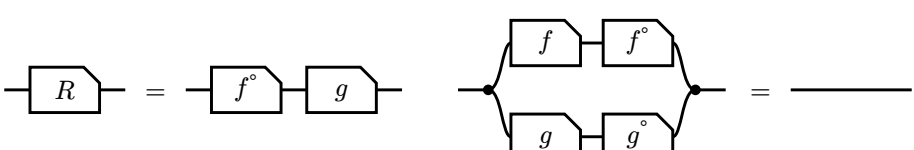


Its universal property, `dom_UP` (AOP/A4_2.lean, B&dM 4.11): among coreflexives, `dom` R is the least one that R factors through.

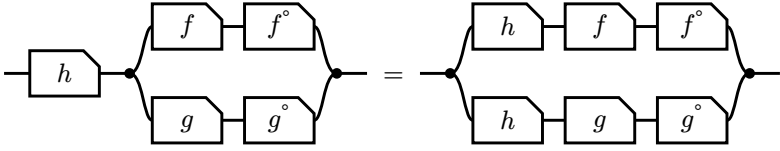


6. Tabulations — `Tabulates`, `tabulates_monnic_pair`

`Freyd/S2_10.lean §2.14`: maps $f : c \rightarrow a$ and $g : c \rightarrow b$ **tabulate** $R : a \rightarrow b$ when $R = f^\circ \gg g$ and $f \gg f^\circ \cap g \gg g^\circ = 1$. The first equation says the span computes R ; the second says the span is **jointly monic** — nothing collapses.



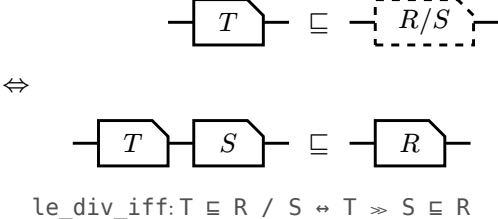
§2.141, `tabulates_monnic_pair`: that second equation makes (f, g) a monic pair in `Map(A)`. The proof is one rewrite — push a map h through the meet, which is legal by `simple_dist_inter` above, since every map is simple.



Read left to right, the left-hand side is $h \gg 1 = h$; read right to left, the right-hand side is built only from $h \gg f$ and $h \gg g$. So two maps agreeing on f and on g are equal.

7. Division as a two-sided rule — `le_div_iff`

`Freyd/S2_30.lean §2.31`: $T \sqsubseteq R / S \leftrightarrow T \gg S \sqsubseteq R$. Both sides of the equivalence are drawable today, but R / S itself is **not** a composite of the generators — every operation of this calculus is monotone in each hole, and division is antitone in S . It is drawn dashed for that reason; a first-class residual box needs the second composition of `diag/F0.lean` (`DiagrammaticAlgebraOfFirstOrderLogic.pdf §5`), which is phase 9.



Taking $T := R / S$ in the right-to-left direction gives the semi-commutative triangle that defines division, `div_comp_le`:

